

LLM SCA How far are we?

Abstract

1 Motivation

The proliferation of Large Language Models (LLMs) has been dramatically accelerated by collaborative platforms like the [Hugging Face Hub](#), which serves as a central repository for model sharing. A cornerstone of this ecosystem is the iterative development of new models by building upon existing ones through techniques such as fine-tuning, merging, and quantization. This process creates a complex web of inter-model dependencies, which we define as **model genealogy** or **lineage**. A clear understanding of a model's genealogy is paramount for ensuring traceability, reproducibility, and compliance, as derivative models often inherit biases, vulnerabilities, or licensing constraints from their predecessors.

However, the mechanisms currently available on the Hugging Face Hub for documenting this critical lineage information are frequently inconsistent, incomplete, or entirely absent. This lack of a systematic and enforceable standard for provenance documentation poses a significant barrier to researchers and developers. The most visible symptom of this issue is the difficulty, or outright impossibility, of programmatically constructing a model's lineage—a structure ideally represented as a “Model Tree”. This is not a superficial interface problem but a direct consequence of fundamental flaws in how provenance data is recorded at its source.

To illustrate the severity of this challenge, we will dissect two distinct modes of documentation failure, each representing a progressively worsening degree of information obfuscation.

1.1 Problem 1: Lineage Concealed in Unstructured Text

In the most common scenario, lineage information exists but is locked within free-form text, rendering it inaccessible to automated tools. This forces researchers into a manual, error-prone process of discovery.

Case Study 1: DavidAU/NSFW_DPO_Noromaid-7b-GGUF

The Hugging Face repository of this model does not include a model tree; however, its model card explicitly states:

NSFW_DPO_Noromaid-7b-Mistral-7B-Instruct-v0.1 is a merge of the following models: mistralai/Mistral-7B-Instruct-v0.1 and athirdpath/NSFW_DPO_Noromaid-7b.

Notably, the Hugging Face pages of both referenced models in the model card contain their respective model trees. This reliance on unstructured documentation makes the automated construction of a lineage graph infeasible. It imposes a significant manual burden for identifying even a direct parent and completely obstructs any large-scale, systematic analysis of the model ecosystem.

Case Study 2: timdettmers/guanaco-13b

Another example is the timdettmers/guanaco-13b model. While this model is a fine-tuned version of LLaMA, this pivotal relationship is not captured in any structured, machine-readable metadata. To discover this link, a user must manually navigate to the model's repository, open the “Model Card” (a Markdown file), and parse through natural language prose to find the statement:

Guanaco models are open-source finetuned chatbots obtained through 4-bit QLoRA tuning of LLaMA base models...

1.2 Problem 2: Lineage Outsourced to Fragile External Links

A more precarious situation arises when provenance information is not even located within the Hugging Face platform, but is instead hyperlinked to an external, third-party source.

Case Study 3: pfnet/plamo-2.1-2b-cpt

The model's homepage lacks a model tree, and its model card merely states that the model was derived by reducing the parameter size from an 8B-parameter model, without explicitly specifying the base model used for this operation. However, the model card provides links to three technical blogs for reference. By following these external links, we can infer that the mentioned 8B-parameter model is plano-2-8b.

Case Study 4: Remilistrasza/Mysterious_SDXL_Lah

Consider the Remilistrasza/Mysterious_SDXL_Lah model. Its Model Card is starkly empty of descriptive content, offering only a single hyperlink to a page on [Civitai](#), another model-sharing platform. To identify the base model, a user is forced to leave the Hugging Face ecosystem. Only upon navigating to this external page does one discover that the base model is “SDXL 1.0”.

This practice introduces two severe risks: **information fragmentation** and **link rot**. The integrity and availability of the lineage information become wholly dependent on the uptime and archival policies of an external website. Should the link become invalid (a 404 error), or the content on the remote page be altered or deleted, the model's traceable lineage is permanently severed. This practice renders the model's provenance fragile and unverifiable from within the Hugging Face ecosystem.

These documented failures, which range from information concealed in prose to its complete omission, demonstrate a clear and pressing need for an automated and reliable method to construct

model genealogies. The current reliance on inconsistent and manual documentation is an unsustainable bottleneck in an ecosystem that is scaling rapidly. To bridge this critical gap, we propose an approach to automatically identify and verify the derivative relationships between models, thereby enabling robust and scalable lineage tracing.

2 Benchmark

2.1 Construction

The construction of our benchmark involves a systematic pipeline that traces model derivations from base models to their descendants on Hugging Face. This process is divided into five key steps:

Step 1: Obtaining Base Model List. We begin by collecting a list of popular base models for the text-generation task from Hugging Face. Utilizing the `HfApi` provided by `huggingface_hub`, we retrieve the top 50 non-gated models sorted by the number of likes. The model identifiers are stored in a file named `base_models.txt`, where each line corresponds to one base model ID. This step is implemented via **Script 1**.

Step 2: Building the Model Derivation Tree. Based on `base_models.txt`, we recursively crawl derivative models to construct a “model forest”. For each base model, we explore its derivatives up to four levels deep, covering four derivation types: finetune, adapter, quantized, and merge. For each derivation type, a maximum of six sibling nodes are considered. Every node in the tree records metadata such as model name, crawl timestamp, depth level, and its child nodes. The resulting forest structure is serialized into `model_forest.json`. This step is implemented via **Script 2**.

Step 3: Constructing Model Pairs. From the model forest, we extract two categories of model pairs: adjacent parent-child pairs and non-adjacent ancestor-descendant pairs. By performing a Breadth-First Search (BFS) traversal on each tree, we generate:

- **Adjacent Pairs:** Direct parent-child model pairs.
- **Non-Adjacent Pairs:** All ancestor-descendant pairs where the ancestor is not the immediate parent.

These pairs are recorded in `adjacent_pairs.jsonl` and `non_adjacent_pairs.jsonl`, respectively. This step is implemented via **Script 3**.

Step 4: Single-Threaded Model Validation. To ensure data quality, we validate the structural correctness of all adjacent model pairs. For each model, we inspect its file tree on Hugging Face, checking for the presence of critical files such as `config.json`, `adapter_config.json`, and various parameter files (e.g., `.bin`, `.pt`, `.safetensors`, etc.). We verify whether the claimed derivation type aligns with the actual file structure and whether the parent model is valid. The results are compiled into a comprehensive validation report named `model_validation_report.json`. This step is implemented via **Script 4**.

Step 5: Filtering Erroneous Model Pairs. Based on the validation report, we remove all erroneous model pairs from the dataset. Models flagged with any inconsistencies or missing critical files are excluded. The cleaned set of adjacent model pairs is saved in

Table 1: Statistics of the GRANDMOTHER benchmark.

Statistic	Value
#Graphs	31
#Model pairs (edges)	248
Min / Median / Max pairs per graph	1 / 6 / 22
Avg. pairs per graph	8.0
#Models (nodes)	279
#Root models	31
#Intermediate models	44
#Leaf models	204
Share of leaf models	73.1%
Min / Avg / Max models per graph	2 / 9.0 / 23
Avg. out-degree of non-leaf models	3.31
Max fine-tuning depth	2

`adjacent_pairs_without_error.json`. This step is implemented via **Script 5**.

2.2 Characteristics

Our benchmark exhibits several notable characteristics that reflect its diversity and representativeness of the Hugging Face model ecosystem:

- **Variety of Derivation Types:** The dataset covers four major derivation types—finetune, adapter, quantized, and merge—capturing a wide range of model transformation patterns.
- **Hierarchical Depth:** With a recursive crawling depth of up to four levels, the benchmark contains both shallow and deep derivation chains, enabling analysis of both immediate and long-range model inheritance.
- **Scale and Coverage:** Starting from 50 popular base models, the recursive traversal results in a substantial model forest, ensuring coverage of highly influential models as well as their derivative variants.
- **Data Cleanliness:** Through rigorous validation and error filtering, the dataset ensures high structural integrity, making it suitable for tasks that require reliable model lineage information.
- **Real-World Relevance:** The benchmark reflects actual user-driven derivation patterns on Hugging Face, providing insights into community practices such as model fine-tuning trends, adapter usage, and quantization adoption.

3 Empirical Results

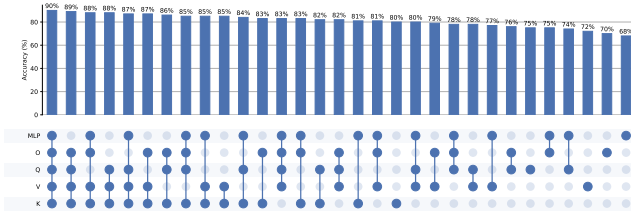
We design experiments to explore the following research questions.

- **Measurement Evaluation (RQ1):** How do different measurement metrics affect the effectiveness of LLM lineage detection?
- **Layers in Transformer Block Evaluation (RQ2):** How do different model layers influence the effectiveness of LLM lineage detection?
- **Failure Analysis (RQ3):** Why do existing methods fail to detect lineage?
- **Integration (RQ4):** using the above findings, the best performance can achieve? (80%)

Table 2: Lineage Detection Accuracy (%) w.r.t. Different Measurement Metrics, Direction Methods, and Graph Algorithms (RQ1)

Graph Algorithm	Dir.	Measurement Metric					
		L2	Cos.	AM	AMx	SA	SR
+ MoTHer	K	***	***	***	***	***	***
	T	***	***	***	***	***	***
+ Hard-Constraint	K	***	***	***	***	***	***
	T	***	***	***	***	***	***
+ Undirected-Mst	K	***	***	***	***	***	***
	T	***	***	***	***	***	***
+ atlas_snake_fan	K	***	***	***	***	***	***
	T	***	***	***	***	***	***

Ablation Study: Impact of Q/K/V/O/MLP on Accuracy

**Figure 1: Accuracy of lineage detection with different combinations of transformer block components (RQ2).**

3.1 Setup

Measurement Metrics. XXX

Accuracy.

Layers. XXX

Graph Algorithms. XX

3.2 RQ1: Measurement Evaluation

Summary: L2+timestamp+xxgraph algorithm performs best.

3.3 RQ2: Layers in Transformer Block Evaluation

Summary: using QKV layer performs best. K layer contributes most.

Summary:

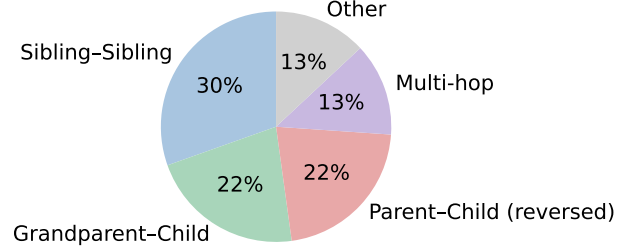
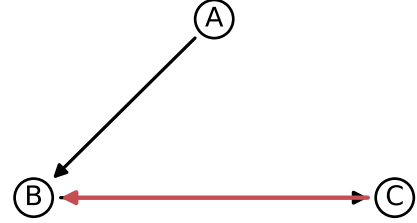
3.4 RQ3: Failure Analysis

Multi-hop Analysis (RQ3): XX

4 Related Work

4.1 Model Signature and Watermark

Determining the provenance and protecting the intellectual property of deep learning models has traditionally relied on watermarking techniques [27]. Early methods introduced watermarks by embedding backdoors into the network during the training phase [1]. While effective for verifying ownership, contemporary watermarking approaches for Large Language Models (LLMs) primarily rely on embedding specific statistical patterns into the generated text to identify ownership from outputs [2, 8, 10, 22]. However, this

**Figure 2: Proportion of different types of erroneous edges in the constructed lineage graph (RQ3).**

Red arrow: erroneous sibling edge (C → B)

Figure 3: Case study illustrating erroneous edges in the constructed lineage graph (RQ3).

inherently alters the output distribution, frequently sacrificing the model’s generation quality and practical utility.

4.2 Model Lineage Analysis

Model Attribution and Similarity Metrics. To avoid the continuous generation degradation inherent to watermarks, active model fingerprinting proactively injects query-response triggers via instruction tuning [15, 21], adversarial prompt suffixes [4], stealthy Chain-of-Thought extractors [14], or output statistics [16]. However, these intrusive modifications remain susceptible to detection, performance degradation, and fine-tuning-induced forgetting, which exacerbates as model scales increase [11]. Consequently, research has shifted toward non-intrusive, post-hoc similarity metrics.

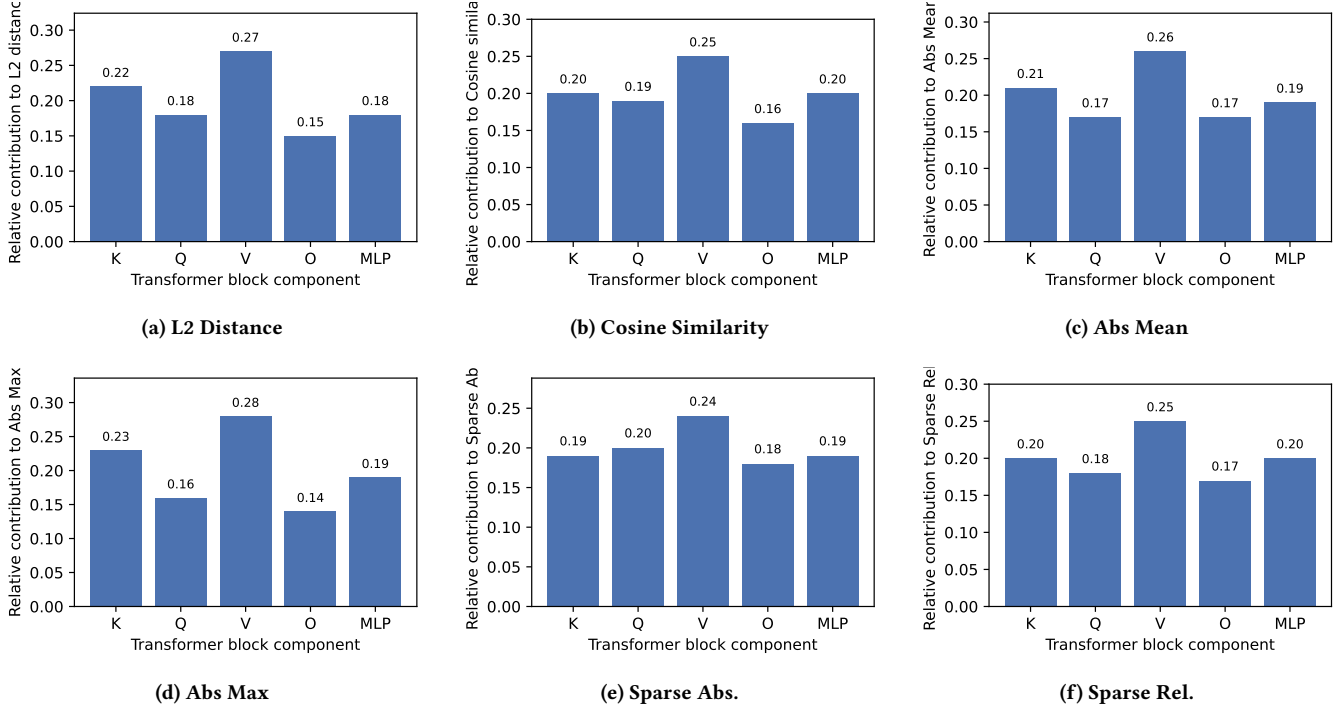


Figure 4: Contribution of different transformer block components (K/Q/V/O/MLP) to various distance metrics (RQ2).

These approaches initially focused on dynamic runtime inference, evaluating pairwise similarity through black-box output distributions [12] or white-box activation and gradient statistics [20, 28]. Because dynamic inference incurs prohibitive forward-pass computational overhead, recent methods transition to static weight analysis. While early static techniques trace full training trajectories for smaller architectures [25], LLM-centric approaches extract cross-layer statistical consistencies [24] or utilize Singular Value Decomposition (SVD) [18]. To further resist adversarial permutation and scaling attacks, state-of-the-art static methods compute robust structural invariants [26] and orthogonal embedding relations [19, 29]. Despite these advancements in extraction efficiency and robustness, all aforementioned metrics are fundamentally limited to strictly pairwise similarity evaluation.

Ecosystem-Scale Model Lineage Reconstruction. Beyond pairwise verification, pioneering works scale similarity metrics to reconstruct macroscopic evolutionary graphs of the open-source ecosystem, aiming to uncover complex N -to- N genealogical trees [13]. Initial black-box methods leverage dynamic inference to compute vocabulary probability differences, such as Nei’s genetic distance, for phylogenetic tree construction [23]. However, relying purely on distance yields unrooted trees that fail to infer evolutionary directionality and often misidentify root models. To resolve this lack of directionality, recent white-box approaches incorporate intrinsic or extrinsic signals. Specifically, they utilize weight-based L_2 distances combined with layer kurtosis [6] or metadata timestamps [5] to construct directed lineages, accurately charting how foundation models adapt and branch over time.

Layer-wise Dynamics in Transformers. Understanding the layer-wise internal mechanisms of Transformers has long been

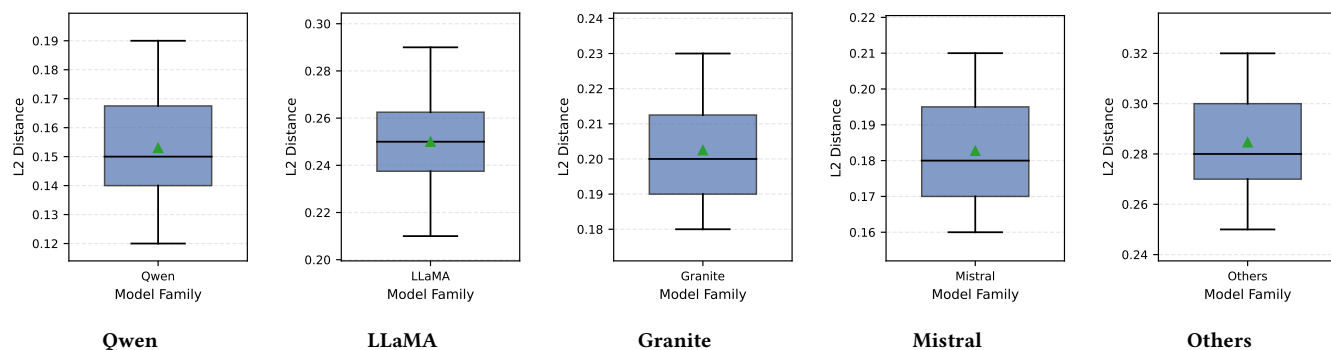
a focal point for model interpretability and architecture optimization. While early studies analyzed the hierarchical progression of linguistic features across hidden layers [9, 17], recent evidence suggests that Transformer components exhibit significant functional specialization and asymmetric sensitivity during model evolution. For instance, Feed-Forward Networks (FFN) are increasingly recognized as key-value memories that store factual knowledge [3], while attention sub-layers (Q, K, V) govern contextual alignment. The success of parameter-efficient fine-tuning (PEFT) further underscores this heterogeneity, demonstrating that updates to specific matrices like W_q and W_v are often sufficient to capture the majority of model shifts [7]. In the context of model provenance, specific weight blocks have been found to maintain invariant spectral signatures during reuse [18], yet existing lineage detection frameworks largely treat all parameters as a homogeneous vector space when calculating inter-model distances. It remains unexplored how individual components—namely the Q, K, V, and MLP matrices—differentially contribute to the fidelity and robustness of macroscopic lineage graph reconstruction.

4.3 Software Supply Chain

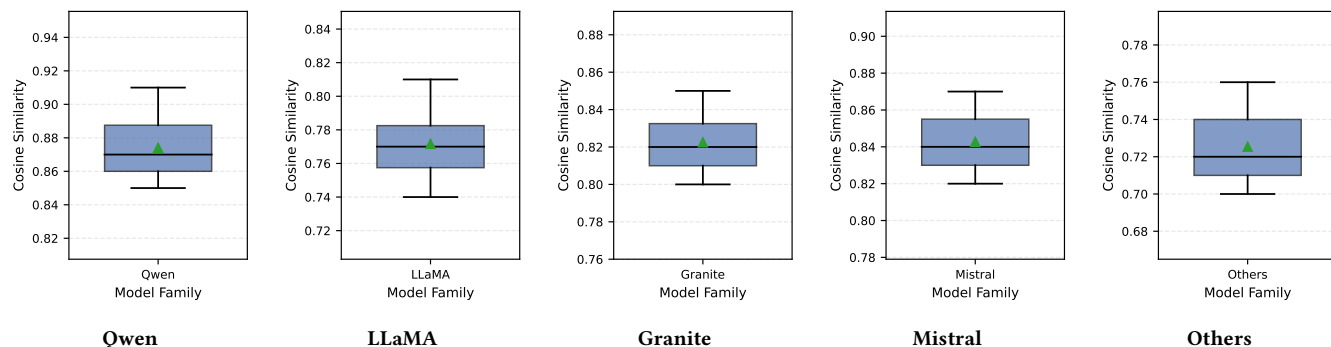
References

- [1] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. 2018. Turning Your Weakness Into a Strength: Watermarking Deep Neural Networks by Backdooring. In *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, William Enck and Adrienne Porter Felt (Eds.). USENIX Association, 1615–1631. <https://www.usenix.org/conference/usenixsecurity18/presentation/adi>
- [2] Miranda Christ, Sam Gunn, and Or Zamir. 2024. Undetectable Watermarks for Language Models. In *The Thirty Seventh Annual Conference on Learning Theory, June 30 - July 3, 2023, Edmonton, Canada (Proceedings of Machine Learning Research, Vol. 247)*, Shipra Agrawal and Aaron Roth (Eds.). PMLR, 1125–1139. <https://proceedings.mlr.press/v247/christ24a.html>

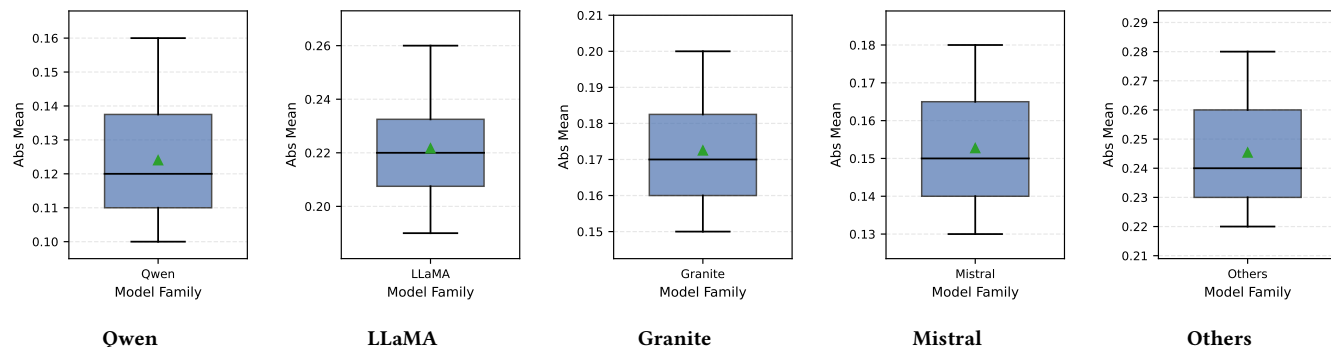
- [3] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 5484–5495. <https://doi.org/10.18653/v1/2021.emnlp-main.446>
- [4] Martin Gubri, Dennis Ulmer, Hwaran Lee, Sangdoo Yun, and Seong Joon Oh. 2024. TRAP: Targeted Random Adversarial Prompt HoneyPot for Black-Box Identification. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11–16, 2024 (Findings of ACL, Vol. ACL 2024)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, 11496–11517. <https://doi.org/10.18653/V1/2024.FINDINGS-ACL.683>
- [5] Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. Charting and Navigating Hugging Face’s Model Atlas. *CoRR abs/2503.10633* (2025). <https://doi.org/10.48550/ARXIV.2503.10633> arXiv:2503.10633
- [6] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2025. Unsupervised Model Tree Heritage Recovery. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=QVj3kUvdv1>
- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022*. OpenReview.net. <https://openreview.net/forum?id=nZeVKeFYf9>
- [8] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A Watermark for Large Language Models. In *International Conference on Machine Learning, ICML 2023, 23–29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 17061–17084. <https://proceedings.mlr.press/v202/kirchenbauer23a.html>
- [9] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey E. Hinton. 2019. Similarity of Neural Network Representations Revisited. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9–15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 3519–3529. <http://proceedings.mlr.press/v97/kornblith19a.html>
- [10] Rohith Kudithipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2024. Robust Distortion-free Watermarks for Language Models. *Trans. Mach. Learn. Res.* 2024 (2024). <https://openreview.net/forum?id=FpaCL1MO2C>
- [11] Anshul Nasery, Jonathan Hayase, Creston Brooks, Peiyao Sheng, Himanshu Tyagi, Pramod Viswanath, and Sewoong Oh. 2025. Scalable Fingerprinting of Large Language Models. *CoRR abs/2502.07760* (2025). <https://doi.org/10.48550/ARXIV.2502.07760> arXiv:2502.07760
- [12] Dario Pasquini, Evgenios M. Korneopoulou, and Giuseppe Ateniese. 2025. LLMmap: Fingerprinting for Large Language Models. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13–15, 2025*, Lujio Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 299–318. <https://www.usenix.org/conference/usenixsecurity25/presentation/pasquini>
- [13] Mohammad Shahedur Rahman, Peng Gao, and Yuede Ji. 2025. HuggingGraph: Understanding the Supply Chain of LLM Ecosystem. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM 2025, Seoul, Republic of Korea, November 10–14, 2025*, Meeyoung Cha, Chanyoung Park, Noseong Park, Carl Yang, Senjuti Basu Roy, Jessie Li, Jaap Kamps, Kijung Shin, Bryan Hooi, and Lifang He (Eds.). ACM, 5997–6005. <https://doi.org/10.1145/3746252.3761510>
- [14] Zhenzhen Ren, Guobiao Li, Sheng Li, Zhenxing Qian, and Xinpeng Zhang. 2025. CoTSRF: Utilize Chain of Thought as Stealthy and Robust Fingerprint of Large Language Models. *CoRR abs/2505.16785* (2025). <https://doi.org/10.48550/ARXIV.2505.16785> arXiv:2505.16785
- [15] Mark Russinovich and Ahmed Salem. 2024. Hey, That’s My Model! Introducing Chain & Hash, An LLM Fingerprinting Technique. *CoRR abs/2407.10887* (2024). <https://doi.org/10.48550/ARXIV.2407.10887> arXiv:2407.10887
- [16] Yun-Yun Tsai, Chuan Guo, Junfeng Yang, and Laurens van der Maaten. 2025. RoFL: Robust Fingerprinting of Language Models. *CoRR abs/2505.12682* (2025). <https://doi.org/10.48550/ARXIV.2505.12682> arXiv:2505.12682
- [17] Betty van Aken, Benjamin Winter, Alexander Löser, and Felix A. Gers. 2019. How Does BERT Answer Questions?: A Layer-Wise Analysis of Transformer Representations. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3–7, 2019*, Wenwu Zhu, Dacheng Tao, Xueqi Cheng, Peng Cui, Elke A. Rundensteiner, David Carmel, Qi He, and Jeffrey Xu Yu (Eds.). ACM, 1823–1832. <https://doi.org/10.1145/3357384.3358028>
- [18] Suqing Wang, Ziyang Ma, Li Xinyi, and Zuchao Li. 2025. Ghost in the Transformer: Detecting Model Reuse with Invariant Spectral Signatures. *arXiv:2511.06390* [cs.CR] <https://arxiv.org/abs/2511.06390>
- [19] Zhaomin Wu, Haodong Zhao, Ziyang Wang, Jizhou Guo, Qian Wang, and Bing-sheng He. 2025. LLM DNA: Tracing Model Evolution via Functional Representations. *CoRR abs/2509.24496* (2025). <https://doi.org/10.48550/ARXIV.2509.24496> arXiv:2509.24496
- [20] Zehao Wu, Yanjie Zhao, and Haoyu Wang. 2025. TensorGuard: Gradient-Based Model Fingerprinting for LLM Similarity Detection and Family Classification. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16–20, 2025*. IEEE, 445–456. <https://doi.org/10.1109/ASE63991.2025.00044>
- [21] Jiashu Xu, Fei Wang, Mingyu Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. 2024. Instructional Fingerprinting of Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 3277–3306. <https://doi.org/10.18653/v1/2024.naacl-long.180>
- [22] Xi Yang, Kejiang Chen, Weiming Zhang, Chang Liu, Yuang Qi, Jie Zhang, Han Fang, and Nenghai Yu. 2023. Watermarking Text Generated by Black-Box Language Models. *CoRR abs/2305.08883* (2023). <https://doi.org/10.48550/ARXIV.2305.08883> arXiv:2305.08883
- [23] Nicolas Yax, Pierre-Yves Oudeyer, and Stefano Palminteri. 2025. PhyloLM: Inferring the Phylogeny of Large Language Models and Predicting their Performances in Benchmarks. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=rTQNGQxm4K>
- [24] Do-hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. 2025. Intrinsic Fingerprint of LLMs: Continue Training is NOT All You Need to Steal A Model! *CoRR abs/2507.03014* (2025). <https://doi.org/10.48550/ARXIV.2507.03014> arXiv:2507.03014
- [25] Rungpeng Yu and Xinchao Wang. 2024. Neural Lineage. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16–22, 2024*. IEEE, 4797–4807. <https://doi.org/10.1109/CVPR52733.2024.00459>
- [26] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. HuRef: HUMAN-REAdable Fingerprint for Large Language Models. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 126332–126362. <https://doi.org/10.52202/079017-4013>
- [27] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph. Stoecklin, Heqing Huang, and Ian M. Molloy. 2018. Protecting Intellectual Property of Deep Neural Networks with Watermarking. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, AsiaCCS 2018, Incheon, Republic of Korea, June 04–08, 2018*, Jong Kim, Gail-Joon Ahn, Seungjoo Kim, Yongdae Kim, Javier López, and Taesoo Kim (Eds.). ACM, 159–172. <https://doi.org/10.1145/3196494.3196550>
- [28] Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2025. REEF: Representation Encoding Fingerprints for Large Language Models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net. <https://openreview.net/forum?id=SnDmPkOj0T>
- [29] Ruichong Zhang. 2025. Matrix-Driven Instant Review: Confident Detection and Reconstruction of LLM Plagiarism on PC. *CoRR abs/2508.06309* (2025). <https://doi.org/10.48550/ARXIV.2508.06309> arXiv:2508.06309



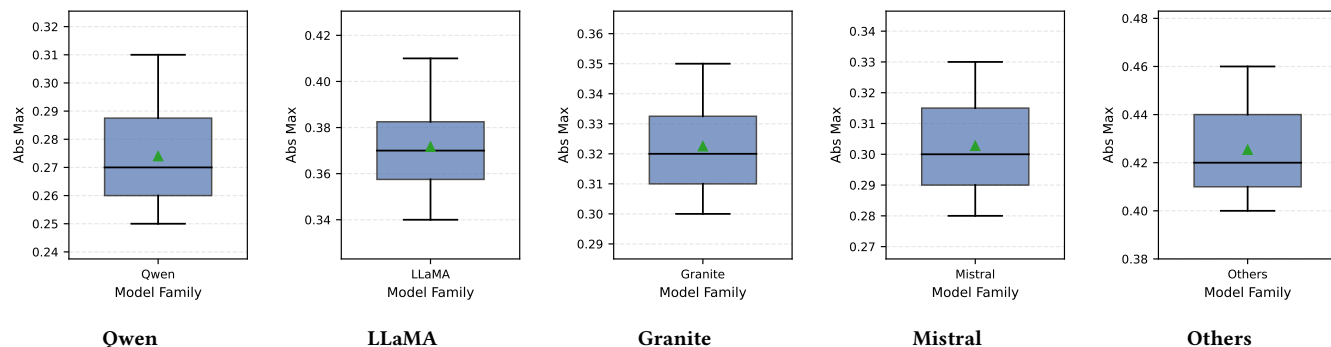
(a) L2 Distance



(b) Cosine Similarity



(c) Abs Mean



(d) Abs Max

